AgenThinkMesh — Full Technical Briefing

Live URL: https://agenthink-7enctkan.manus.space
GitHub: https://github.com/agenthinkai/agenthinkmesh
Stack: React 19 + TypeScript + Tailwind 4 + Express 4 + tRPC 11 + Drizzle ORM + MySQL/TiDB
Auth: Manus OAuth (`protectedProcedure` / `ctx.user`)
Deployment: Manus managed hosting (single-click publish)

---

1. Architecture Overview

The project is a full-stack TypeScript monorepo. There are no separate frontend/backend repos — everything lives under `/home/ubuntu/agenthinkmesh-full`.

```
client/          React 19 SPA (Vite, Tailwind 4, shadcn/ui)
server/          Express 4 + tRPC 11 backend
  _core/         Framework plumbing: OAuth, context, Vite bridge, LLM
helper, env
  routers/       Feature-specific tRPC routers (one file per domain)
drizzle/         Schema + migrations (MySQL/TiDB via Drizzle ORM)
shared/          Shared constants and types
```

All API calls go through tRPC procedures. There are no raw REST routes except for SSE streaming endpoints and file upload endpoints (which require multipart/form-data and cannot go through tRPC).

---

2. Database Schema

All tables are in MySQL/TiDB. Schema is defined in `drizzle/schema.ts`.

Core tables (template-provided)

Table	Purpose
`users`	Auth, roles (admin/user), plan tier, trial tracking, billing fields
`sessions`	JWT session management

Feature tables (built in this project)

Table	Module	Key columns
`deal_screenings`	Deal Screener	`userId`, `dealName`, `dealText`, `result` (JSON), `verdict`, `yesCount`, `noCount`, `confidenceScore`, `tiebreakerTriggered`, `gccVeto`
`deal_screening_rate_limit`	Deal Screener	`userId`, `windowStart`, `count` — tracks 20 screens/hour per user
`intel_analyses`	Intelligence Agent	`userId`, `institution`, `domain`, `aum`, `inputText`, `result` (JSON), `modules`, `lens`, `isInternal`
`intel_tracked`	Intelligence Agent	`userId`, `institution`, `domain`, `aum`, `lastAnalysis`, `lastFetchedContent`, `trackingSource`
`intel_history`	Intelligence Agent	`trackedInstitutionId`, `result`, `diff`, `fetchedContent`
`intel_briefs`	Intelligence Agent	`userId`, `content`, `weekOf`

Plus additional tables for ETF Studio, AdMesh, Social AI, OpenClaw, Insurance, and Rosie Protocol modules (not detailed here — see `drizzle/schema.ts`).

---

3. Module Inventory

3.1 Deal Screener — Council of 10

Route: `/deals`
Files:

- `server/councilEngine.ts` — core engine
- `server/routers/dealScreener.ts` — tRPC procedures
- `server/dealScreenerUploadRoute.ts` — PDF upload (multer + pdf-parse)
- `client/src/pages/DealScreener.tsx` — full Bloomberg-terminal dark UI
- `server/dealScreener.test.ts` — 6 vitest tests covering all verdict paths

What it does:
Takes a deal memo (text or PDF) and runs it through 10 specialist AI personas in parallel using `Promise.allSettled`. Each persona calls Anthropic Claude with a 15-second timeout. If a persona times out or errors, it falls back to `SOFT_NO` with confidence 0.2. Results are aggregated using a consensus algorithm.

10 Personas:

1. GCC Regulatory Compliance Officer (`GCC_REG`)
2. GCC Shariah Compliance Advisor (`GCC_SHARIAH`)
3. CFO / Financial Modeller (`CFO`)
4. CTO / Technical Due Diligence (`CTO`)
5. Market Intelligence Analyst (`MARKET`)
6. ESG & Impact Analyst (`ESG`)
7. Legal & Governance Counsel (`LEGAL`)
8. Operational Risk Manager (`OPS`)
9. Portfolio Construction Strategist (`PORTFOLIO`)
10. Growth & Commercial Analyst (`GROWTH`)

Persona response schema (per persona):

```
{
  "vote": "HARD_YES | SOFT_YES | SOFT_NO | HARD_NO",
  "confidence": 0.0–1.0,
  "rationale": "string (max 400 chars)",
  "key_flags": ["string", "string", "string"]
}
```

Consensus rules (in priority order):

1. If `GCC_REG` or `GCC_SHARIAH` votes `HARD_NO` → **VETOED** (immediate, no override)
2. If `finalYesCount >= 8` AND `hardYesCount >= 6` → **APPROVED**
3. If `finalYesCount >= 8` AND `hardYesCount < 6` → **APPROVED_WITH_CONDITIONS**
4. If `noCount >= 5` AND any `HARD_NO` present → **REJECTED**
5. If `yesCount == 7` AND `noCount == 3` → **TIEBREAKER** triggered: flip the lowest-confidence `SOFT_NO` to `SOFT_YES`, re-evaluate
6. Otherwise → **REJECTED**

Rate limiting: 20 screens/hour per authenticated user (tracked in `deal_screening_rate_limit` table, in-memory window).

PDF upload: `POST /api/deals/upload-pdf` — multer, 5 MB cap, pdf-parse extracts first 1500 chars, injected into `deal_text` if the pasted field is empty.

tRPC procedures:

- `trpc.dealScreener.screen` — protected, calls councilEngine, persists to DB
- `trpc.dealScreener.history` — protected, returns user's deal history
- `trpc.dealScreener.getById` — protected, returns full IC report by dealId
- `trpc.dealScreener.rateLimit` — protected, returns remaining screens in current window

---

3.2 Intelligence Agent

Route: `/intelligence`
Files:

- `server/routers/intelligence.ts` — tRPC procedures
- `server/intelligenceParseRoute.ts` — document parse endpoint
- `client/src/pages/IntelligenceHome.tsx` — main analysis entry point
- `client/src/pages/IntelligenceTracking.tsx` — institution tracking dashboard
- `client/src/pages/IntelligenceBriefs.tsx` — weekly intelligence briefs
- `client/src/pages/IntelligenceHistory.tsx` — analysis history
- `client/src/pages/IntelligenceAdmin.tsx` — admin panel

What it does:
Runs deep institutional intelligence analysis on financial institutions, funds, or companies. Accepts free-text input or uploaded documents. Produces structured intelligence reports covering regulatory posture, financial health, competitive positioning, and risk flags.

3.3 OpenClaw

Route: `/openclaw`
Sub-routes: `/openclaw/discovery`, `/openclaw/bridge`, `/openclaw/policy`, `/openclaw/manifests`
Files: `client/src/pages/openclaw/` (5 pages)

What it does:
GCC-focused legal intelligence platform. Provides contract clause analysis, regulatory discovery, policy interpretation, and manifests management for enterprise legal teams operating in the Gulf Cooperation Council jurisdictions.

3.4 AdMesh

Route: `/admesh`
Files: `client/src/pages/admesh/`, `server/routers/admesh.ts`, `server/admeshStreamRoute.ts`

What it does:
AI-powered advertising creative intelligence for GCC markets. Generates Arabic/English ad briefs, 10 ad card variants, storyboards, and campaign strategy documents. Streams results via SSE.

3.5 Social AI

Route: `/social`
Files: `client/src/pages/social/`, `server/routers/socialMedia.ts`, `server/socialMediaStreamRoute.ts`

What it does:
Multi-platform social media content intelligence. Supports Arabic/English dialect localisation, cross-platform publishing strategy, and content calendar generation. Streams results via SSE.

3.6 Insurance & Reinsurance Intelligence

Route: `/insurance`
Files: `client/src/pages/insurance/`, `server/routers/insurance.ts`, `server/insuranceStreamRoute.ts`

What it does:
GCC insurance and reinsurance market intelligence. Analyses policy structures, reinsurance treaties, regulatory compliance (SAMA, CBUAE), and actuarial risk profiles.

3.7 ETF Launch Studio

Route: `/etf`
Files: `client/src/pages/etf/`, `server/etfRoute.ts`, `server/routers/` (ETF

procedures)

What it does:
End-to-end ETF product design and launch workflow. Covers index construction, regulatory filing preparation, market maker selection, and GCC-specific Shariah screening.

3.8 Rosie Protocol

Route: `/rosie`
Files: `client/src/pages/rosie/`, `server/routers/` (Rosie procedures)

What it does:
Healthcare AI intelligence platform. Clinical protocol analysis, drug interaction screening, and GCC healthcare regulatory compliance.

3.9 Agent Registry

Route: `/registry`
Files: `client/src/pages/Registry.tsx`, `server/agentRoutes.ts`

What it does:
Catalogue of all 125 specialist agents deployed on the platform. Shows agent name, domain, status (Active/Standby/Ready), and health. Agents are health-checked on a cron schedule via `server/jobs/healthCheck.ts`.

---

4. PDF Report Generation

File: `server/pdfReport.ts`
Library: PDFKit

All modules that produce structured reports can export them as PDF. The PDF generation system:

- Paints a dark navy background (`#080D1A`) on **every page** via the `pageAdded` event listener (not just page 1 — this was a bug that was fixed)

- Draws a full branded header on page 1 (large logo, task metadata, confidence score)
- Draws a slim continuation header on pages 2+ (AgenThinkMesh · Task ID · Task Type · Date on left, Page N on right, cyan divider line)
- Uses Inter for body text and JetBrains Mono for code/data fields
- All text is white (`#F0F4FA`) or cyan (`#38BDF8`) on the dark background

---

5. LLM Integration

File: `server/_core/llm.ts`
Helper: `invokeLLM({ messages, response_format?, tools? })`

All LLM calls go through `invokeLLM` on the server side. The helper:

- Uses the `ANTHROPIC_API_KEY` environment variable (added to `server/_core/env.ts`)
- Returns the raw Anthropic API response
- Is called from tRPC procedures only — never from client-side code

For streaming responses (AdMesh, Social AI, Insurance, ETF), SSE routes call Anthropic's streaming API directly and pipe chunks to the client via `res.write()` .

---

6. Auth & Access Control

- **Manus OAuth** handles login. The callback is at `/api/oauth/callback` .
- `protectedProcedure` injects `ctx.user` — all feature procedures use this.
- `publicProcedure` is used only for the agent registry list and public landing data.
- The `users` table has a `role` field (`admin` | `user`). Admin-only procedures check `ctx.user.role === 'admin'` .
- Frontend reads auth state via `useAuth()` hook — no manual cookie handling.

---

7. Environment Variables

All secrets are injected via the Manus platform. Key variables:

Variable	Used by
`ANTHROPIC_API_KEY`	councilEngine.ts, all LLM calls
`DATABASE_URL`	Drizzle ORM (MySQL/TiDB)
`JWT_SECRET`	Session cookie signing
`NEWS_API_KEY`	Intelligence Agent (news fetching)
`VITE_APP_ID`	Manus OAuth
`BUILT_IN_FORGE_API_KEY`	Manus built-in APIs (server-side)
`RESEND_API_KEY`	Email drip campaigns

---

8. Testing

Framework: Vitest
Test files: `server/*.test.ts`

Key test file: `server/dealScreener.test.ts` — 6 tests covering all 6 consensus verdict paths:

1. 8 YES with 6+ HARD_YES → APPROVED
2. 8 YES with < 6 HARD_YES → APPROVED_WITH_CONDITIONS
3. GCC_REG votes HARD_NO → VETOED
4. 5+ NO with HARD_NO present → REJECTED
5. Broken JSON from persona → fallback to SOFT_NO
6. Tiebreaker (7 YES / 3 NO) → flip lowest-confidence SOFT_NO → re-evaluate

Mock pattern: `vi.hoisted()` is used to create `mockCreate` before `vi.mock('@anthropic-ai/sdk')` hoisting runs, since the Anthropic client is

instantiated at module level in `councilEngine.ts`.

Total tests: 140 passing across 10 test files. tsc: 0 errors.

---

9. Key Design Decisions

Why tRPC instead of REST for Deal Screener:
The existing platform uses tRPC end-to-end. Adding a REST endpoint would create a second API pattern, break type safety, and require a separate Axios/fetch client. tRPC gives typed inputs/outputs, automatic error handling, and React Query integration for free.

Why per-user rate limiting instead of per-IP:
The platform has auth. IP-based limiting is trivially bypassed with a VPN. Per-user limiting is accurate, tied to the session, and consistent with the user-scoped history model.

Why `Promise.allSettled` instead of `Promise.all` for the 10 persona calls:
`Promise.all` fails fast — one persona timeout kills the entire screening. `Promise.allSettled` lets all 10 run independently. Failed/timed-out personas fall back to `SOFT_NO` rather than crashing the request.

Why 400-char rationale + `key_flags` array instead of 200-char rationale:
A 200-char rationale on a complex deal memo produces one sentence. An IC chair needs to see the 3 specific things each lens flagged, not a summary. `key_flags: [string, string, string]` (max 3 items) gives the IC exactly that — scannable, specific, actionable.

Why `pageAdded` event for PDF backgrounds:
PDFKit's `addPage()` creates a blank white page. The only reliable way to paint a background on every page — including pages triggered by content overflow — is to listen to the `pageAdded` event and draw the background rect immediately, before any content is placed.

---

10. File Count & Size

- **Total TypeScript/TSX files:** ~180
- **Total lines of code:** ~45,000
- **Client pages:** 40+
- **Server routers:** 12 feature routers + main appRouter
- **Database tables:** 20+
- **GitHub repo:** https://github.com/agenthinkai/agenthinkmesh (private)

Generated: March 25, 2026